



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant Teaching
Professor
Lisa Yan

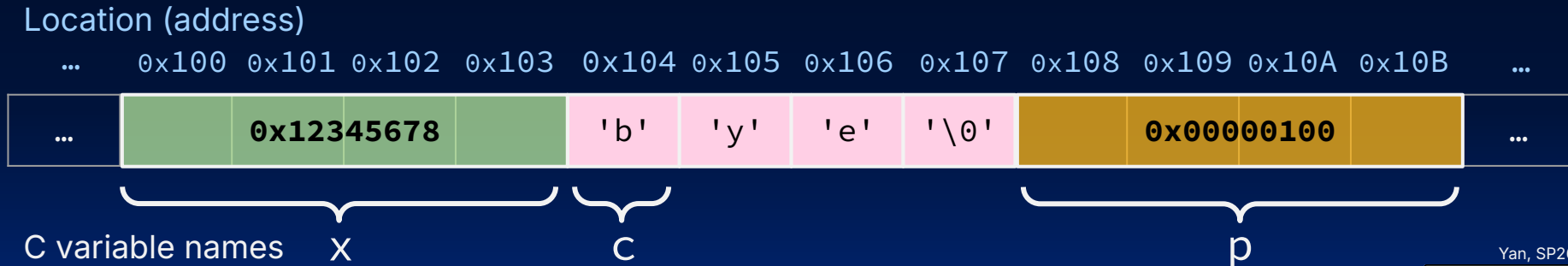
C Pointers, Arrays, and Strings



Memory is a Single Huge Array

- Consider memory to be a byte-addressed array.
 - Each cell of the array has an **address** associated with it.
 - Each cell also stores some **value**.
 - For now, with this abstraction, let's us think we have access to ∞ memory, numbered from 0...

Addresses are commonly written in hexadecimal format.



From Last Time

```
1 #include <stdio.h>
2 int main(int argc, char *argv[]) {
3     int x = 0;
4     int y;
5
6     printf("before: x=%d, y=%d\n",
7           x, y);
8     x++;
9     y += x; // y = y + x
10    printf(" after: x=%d, y=%d\n",
11           x, y);
12    return 0;
13 }
```



Print output:

```
before: x=0, y=22621
after:  x=1, y=22622
```

%d Placeholder for argument, where
d: format value as decimal numeral



Agenda

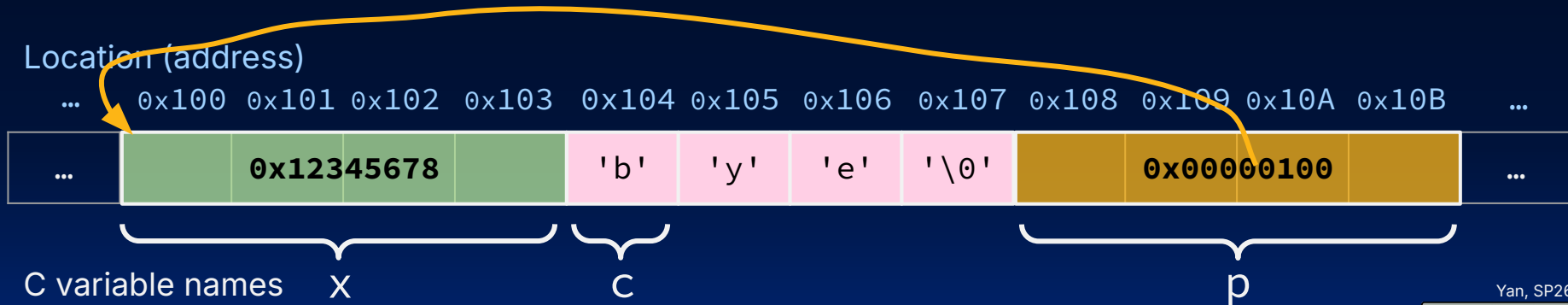
Pointers

- Pointers
- Using Pointers Effectively
- Arrays, Pointer Arithmetic
- ~~Array are Primitive~~
- ~~Strings~~



Pointers store addresses

- **Pointer**: A variable that contains an address.
 - In other words, p **"points"** to a memory location.
- **dereference**, i.e., **follow** pointer
 - Access the value "pointed" to by p





Pointer Syntax

0x100

p

???

0x104

x

3

```
1 int *p;  
2 int x = 3;  
  
3 p = &x;  
  
4 printf("p points to %d\n",  
        *p);  
  
5 *p = 5;
```

- Declaration
- Tells compiler that **variable p is address** of an int



Pointer Syntax



```
1 int *p;  
2 int x = 3;  
3 p = &x;  
4 printf("p points to %d\n",  
        *p);  
5 *p = 5;
```

- Declaration
- Tells compiler that variable `p` is address of an int
- Tells compiler to assign **address of `x`** to `p`
- `&`: "**address operator**" in this context

Pointer Syntax



1 `int *p;`

2 `int x = 3;`

3 `p = &x;`

4 `printf("p points to %d\n",
 *p);`

5 `*p = 5;`

- Declaration
- Tells compiler that variable `p` is address of an `int`
- Tells compiler to assign **address of `x`** to `p`
- `&`: "**address operator**" in this context
- Gets **value pointed to** by `p`
- `*`: "**dereference operator**" in this context

Pointer Syntax



1 `int *p;`

2 `int x = 3;`

3 `p = &x;`

4 `printf("p points to %d\n",
 *p);`

5 `*p = 5;`

- Declaration
- Tells compiler that variable `p` is address of an `int`
- Tells compiler to assign **address of `x`** to `p`
- `&`: "**address operator**" in this context
- Gets value pointed to by `p`
- `*`: "**dereference operator**" in this context
- Changes **value pointed to** by `p`
- Use deref operator `*` on left of `=`

The "star"



The "*" is used in two ways:

Declaration (L1): Indicate p is a pointer

Dereference (L4,L5): Value pointed to by p

```

1 int *p;
2 int x = 3;

3 p = &x;

4 printf("p points to %d\n",
      *p);

5 *p = 5;
  
```

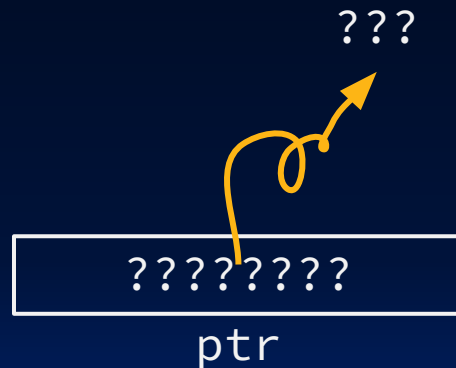
- **Declaration**
- Tells compiler that variable p is address of an int
- Tells compiler to assign **address of x** to p
- &: "**address operator**" in this context
- Gets **value pointed to** by p
- *: "**dereference operator**" in this context
- Changes **value pointed to** by p
- Use deref operator * on left of =



Common C Bug: Garbage Addresses

- Declaring a pointer just allocates space to hold the pointer.
 - It does not allocate something to be pointed to!
- Recall: Local variables in C **are not initialized**.
 - They may contain anything.

```
void f() {  
    int *ptr;  
    *ptr = 5;  
}
```





Agenda

Using Pointers Effectively

- Pointers
- Using Pointers Effectively
- Arrays, Pointer Arithmetic
- ~~Array are Primitive~~
- ~~Strings~~



Pointers in C ... The Good, the Bad, and the Ugly

- To pass a large struct or array to a function, it's **easier/faster/etc.** to pass a pointer.
 - Otherwise, we'd need to copy a huge amount of data!
 - More important when C was invented—compilers didn't produce efficient code in the early 1970s.
 - Nowadays, still used for low-level system code
 - And “pass-by-reference” implementation of object paradigms in other languages.

[at home] Read notes for struct and struct pointer examples



Pointers in C ... The Good, the Bad, and the Ugly

⚠ So, what are the drawbacks?

- Pointers are probably the **single largest source of bugs** in C.

Be careful!

- Most problematic with dynamic memory management
 - Dangling references and memory leaks
- } (more later)



Pointers are Useful When Passing Parameters

C is **pass-by-value**: A function parameter gets assigned **a copy of the argument value**.

Changing the function's copy cannot change the original.

```
void addOne (int x) {  
    x = x + 1;  
}
```

x 3 4

```
int y = 3;  
addOne(y);
```

y 3

▲ y is still 3...



Pointers are Useful When Passing Parameters

C is **pass-by-value**: A function parameter gets assigned **a copy of the argument value**.

Changing the function's copy cannot change the original.

```
void addOne (int x) {  
    x = x + 1;  
}
```

x 3 4

```
int y = 3;  
addOne(y);
```

y 3

⚠ y is still 3...

To get a function to change a value, **pass in a pointer**.

```
void addOne (int *p) {  
    *p = *p + 1;  
}
```

p 0x100

0x100

y 3 4

✓ y is now 4!



NULL pointers...

The pointer of all 0s is special.

- The **NULL** pointer, like in Java, Python, etc...

```
char *p = NULL;  
p 0x00000000
```

If you write to or read from a null pointer, your program should **crash**.

Since **0 is false**, its very easy to do tests for null:

- `if(!p) { /* p is a null pointer */ }`
- `if(q) { /* q is not a null pointer */ }`

```
q 0x104
```

Pointers to Different Data Types

- Pointers are used to point to a value of a particular data type.
 - Normally a pointer can only point to one type.
 - (next week) generic pointers (`void *`)
 - (next week) function pointers
- Just like before: A variable's type helps the compiler determine **how to translate the program to machine code** designed for the computer's architecture:
 - Now with values accessed by the dereference operator!
 - **How many bytes** `*xptr` takes up in memory, and
 - **What operators** `*xptr` supports, etc.

```
int *xptr;  
char *str;  
struct llist *foo_ptr;
```



Agenda

Arrays, Pointer Arithmetic

- Pointers
- Using Pointers Effectively
- Arrays, Pointer Arithmetic
- ~~Array are Primitive~~
- ~~Strings~~



A C array is really just a big block of memory

Declaration:

- `int arr[2];`
- ...declares a 2-element integer array



arr

Declaration and initialization

- `int arr[] = {795, 635};`
- declares and fills a 2-elt integer array



arr

Accessing elements:

- `arr[num]` returns the numth element.
- Shorthand for **pointer arithmetic** + dereference!

`arr[0]; // 795`

Square Indexing = Pointer Arithmetic + Dereference

Pointer arithmetic:

$\text{ptr} + n$

$\text{ptr} - n$

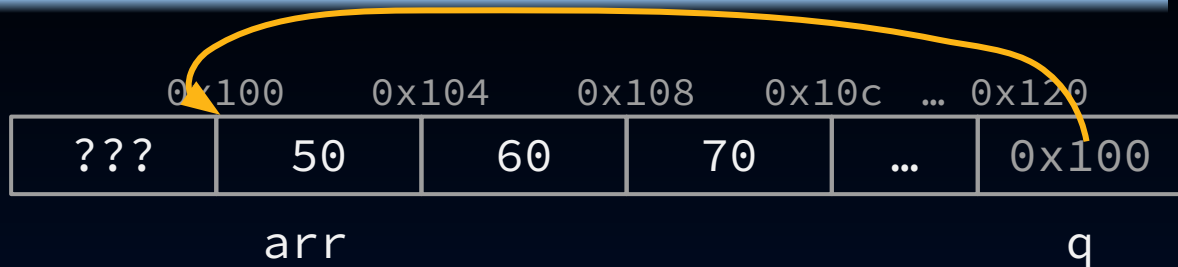
Add (or subtract)

$n * \text{sizeof}(*\text{ptr})$ to (from)
memory address.

Equivalent, i.e.,

"syntactic sugar":

$\text{ptr}[i] \Leftrightarrow *(\text{ptr} + i)$



```
uint32_t arr[] = {50, 60, 70};
uint32_t *q = arr;
```



Pointer Arithmetic

Pointer arithmetic:

$ptr + n$

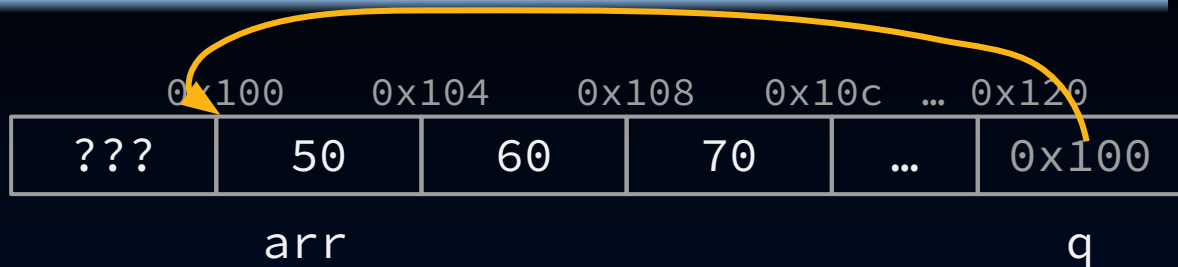
$ptr - n$

Add (or subtract)

$n * \text{sizeof}(*ptr)$ to (from)
memory address.

Equivalent, i.e.,
"syntactic sugar":

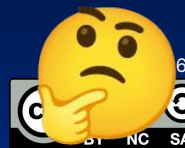
$ptr[i] \Leftrightarrow *(ptr+i)$



```
uint32_t arr[] = {50, 60, 70};  
uint32_t *q = arr;
```

```
printf("    *q: %d is %d\n", *q, q[0]);  
printf("*(q+1): %d is %d\n", *(q+1), q[1]);  
printf("*(q-1): %d is %d\n", *(q-1), q[-1]);
```

```
*q: 50 is 50  
*(q+1): 60 is 60  
*(q-1): ??? is ???
```





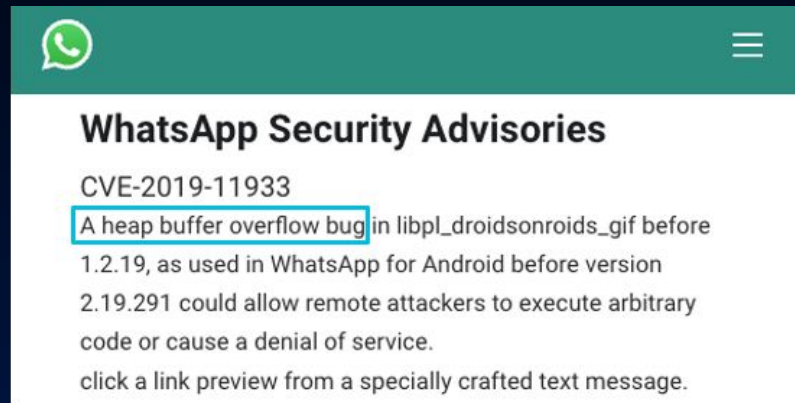
GOT TO HERE

Array Bounds, in General

- Array bounds are **not** checked during element access.
 - Consequence: We can accidentally access off the end of an array
- Improper access may...
 - Corrupts other parts of program, Including internal C data
 - May cause crashes much later

```
int N = 100;  
int foo[N];  
....  
for(int i = 0; i <= N; ++i) {  
    foo[i] = 0;  
}
```

will write past
end of array



Buffer overflow